

Article

Enabling Humans and AI Systems to Retrieve Information from System Architectures in Model-Based Systems Engineering

Vincent Quast , Georg Jacobs , Simon Dehn  and Gregor Höpfner 

Institute of Machine Elements and Systems Engineering, RWTH Aachen University, 52062 Aachen, Germany; georg.jacobs@imse.rwth-aachen.de (G.J.); simon.dehn@imse.rwth-aachen.de (S.D.); gregor.hoepfner@imse.rwth-aachen.de (G.H.)

* Correspondence: vincent.quast@imse.rwth-aachen.de

Abstract

The complexity of modern cyber-physical systems is steadily increasing as their functional scope expands and as regulations become more demanding. To cope with this complexity, organizations are adopting methodologies such as model-based systems engineering (MBSE). By creating system models, MBSE promises significant advantages such as improved traceability, consistency, and collaboration. On the other hand, the adoption of MBSE faces challenges in both the introduction and the operational use. In the introduction phase, challenges include high initial effort and steep learning curves. In the operational use phase, challenges arise from the difficulty of retrieving and reusing information stored in system models. Research on the support of MBSE through artificial intelligence (AI), especially generative AI, has so far focused mainly on easing the introduction phase, for example by using large language models (LLMs) to assist in creating system models. However, generative AI could also support the operational use phase by helping stakeholders access the information embedded in existing system models. This study introduces an LLM-based multi-agent system that applies a Graph Retrieval-Augmented Generation (GraphRAG) strategy to access and utilize information stored in MBSE system models. The system's capabilities are demonstrated through a chatbot that answers questions about the underlying system model. This solution reduces the complexity and effort involved in retrieving system model information and improves accessibility for stakeholders who lack advanced knowledge in MBSE methodologies. The chatbot was evaluated using the architecture of a battery electric vehicle as a reference model and a set of 100 curated questions and answers. When tested across four large language models, the best-performing model achieved an accuracy of 93 percent in providing correct answers.

Keywords: model-based systems engineering (MBSE); artificial intelligence; digital engineering; data engineering; cyber-physical systems; GraphRAG; AI agent



Academic Editor: Melanie King

Received: 28 November 2025

Revised: 6 January 2026

Accepted: 9 January 2026

Published: 12 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the [Creative Commons](https://creativecommons.org/licenses/by/4.0/)

[Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Traditional systems engineering (SE) relies on a document-centric approach in which engineering knowledge is captured and exchanged through text documents, spreadsheets, and informal communication channels. While document-centric approaches may work for small and simple systems, they become less effective as system size and complexity increase, often resulting in fragmented information, inconsistencies, and difficulties in maintaining traceability across multiple disciplines [1].

In parallel, the nature of engineered systems has evolved significantly. Modern products are increasingly developed as cyber–physical systems (CPSs), which tightly integrate software, electronics, and mechanical components into interconnected systems [2]. The development process of CPSs is characterized by a rapidly growing functional scope, distributed development teams, and stricter requirements. These characteristics increase system complexity and place high demands on coordination and information management.

Model-based systems engineering (MBSE) has emerged as a response to these challenges. In contrast to document-centric SE, MBSE establishes centralized system models as the authoritative sources of information. The literature highlights several potential advantages of MBSE, including improved traceability, automated consistency checks, and enhanced collaboration across disciplines [3]. However, although many studies report the perceived benefits of MBSE, only a small fraction have empirically measured these improvements [4].

Additionally, the adoption of MBSE faces challenges during both its introduction and operational use. In the introduction phase, barriers include the high initial modeling effort, steep learning curves, and limited management commitment. In the operational use phase, challenges arise from managing model complexity and retrieving relevant information from large and continuously growing system models [5,6].

Advances in artificial intelligence (AI), particularly in generative AI such as large language models (LLMs), offer promising opportunities to enhance human interaction with system models through natural language. This potential is further amplified by the introduction of the System Modeling Language V2 (SysMLv2), the successor to the widely adopted system modeling language SysML. A key advantage of SysMLv2 lies in its support for textual representation, which enables LLMs to directly generate SysMLv2 code. Recent research explored the potential of LLMs to support the adoption of MBSE. However, our investigation shows that research primarily focuses on the introduction phase by AI-supported model generation, requirement analysis, and automated consistency checking [7–10]. Despite some proofs of concept [11], far less attention has been paid to the operational use phase and facilitating the use of information stored in MBSE models. Addressing this gap is crucial to realizing the full potential of MBSE.

This study aims to develop a methodology that enables AI systems to access and leverage information stored in MBSE system models. The methodology is implemented in an LLM-based multi-agent system that employs a Graph Retrieval-Augmented Generation (GraphRAG) strategy. To demonstrate the feasibility of this approach, a chatbot is developed to answer content-related questions about the underlying system architecture.

The study is structured as follows: After the introduction in Section 1, Section 2 shows the state of the art in generative AI in MBSE, particularly the state of the art in information retrieval in system architectures. Additionally, a general overview of GraphRAG is given. Section 3 introduces the LLM-based multi-agent system. Section 4 shows the results demonstrated on a system architecture of a battery electric vehicle and a question-and-answer set of 100 questions. The study concludes with a discussion and outlook.

2. State of the Art

In this state-of-the-art review, we discuss the benefits and adoption challenges of MBSE, review current research on the use of generative AI in the context of MBSE, and highlight the need for research on enabling both human stakeholders and AI applications to access and leverage information stored in MBSE system models.

2.1. Model-Based Systems Engineering

The International Council of Systems Engineering (INCOSE) defines model-based systems engineering as “a formalized application of modeling to support system requirements, design, analysis, verification and validation activities beginning in the conceptual design phase and continuing throughout development and later lifecycle phases” [12].

MBSE consists of three key pillars: a modeling language, a modeling methodology, and a supporting tool. Traditionally, the modeling language SysML has been primarily graphical, requiring dedicated modeling tools with graphical editors to create and manipulate system models. The latest version, SysMLv2, introduces textual notation alongside the graphical representation. This textual notation enables models to be developed within standard integrated development environments (e.g., Visual Studio Code 1.107.1) [13]. This reduces the dependency on specialized modeling tools and increases flexibility in model creation, as engineers can now choose between graphical, textual, or hybrid approaches. Most MBSE methodologies follow the Requirements–Functional–Logical–Physical (RFLP) paradigm, which is aligned with the left side of the V-Model—a widely established framework for structuring engineering processes starting at stakeholder requirements and breaking them down to physical components [14].

The research community outlines several perceived benefits of MBSE. These perceived benefits include better communication, increased traceability, better complexity management, improved consistency, and increased capacity for reuse, i.e., reusability of models [4] (p. 60). However, Salado et al. investigated that only a small proportion (2 out of 360 or 0.6%) of scientific papers (journals and conference proceedings) actually measured the benefits of MBSE [4] (p. 65).

Moreover, MBSE faces some adoption challenges. In 2015 Bonnet et al. identified the following obstacles in a workshop dedicated to operational deployment of MBSE: cultural resistance, lack of management commitment, tooling challenges, restrictive IT policies and limited support, a steep learning curve, and difficulties in measuring [Return on Investment] ROI [15] (pp. 511–512). Similar adoption challenges were identified by Bayer T. et al. in 2021 [16] (p. 10).

In 2018, Chami et al. summarized that the main challenges in MBSE adoption stem from human and technological factors such as awareness, resistance to change, and lack of clear resources—requiring early executive sponsorship and upfront investment. While some challenges emerge later, like complexity management, they should be anticipated from the start, though this is rarely carried out in practice. Ultimately, organizations must prioritize challenges by importance, timing, and dependencies [5].

In 2024, Call et al. concluded that perceived mental effort, limited access to modeling tools, and poor compatibility with existing SE practices significantly hinder MBSE adoption, underscoring the need to lower entry barriers and better align MBSE with established approaches [17] (p. 473).

2.2. Generative Artificial Intelligence in MBSE

To enhance the accessibility of MBSE, ongoing research explores the application of generative artificial intelligence in the domain of MBSE.

Dumitrescu et al. introduce a maturity model for AI-based assistance systems in model-based systems engineering following the prominent SAE-Levels of automated driving. Level 0 means that no generative AI is used in the development process. Level 1 includes generic AI capabilities to support general tasks. The release of GPT-3.5 by OpenAI in November 2022 has effectively made Level 1 accessible to everybody. Level 2 refers to dedicated engineering copilots that can automate specific development tasks. From level 0 to level 5, the degree of AI assistance grows gradually. Level 5 is described as AI assistants

that can perform planning, execution, and development tasks fully independently. Human stakeholders are proactively informed and alerted to decision needs. Additionally, the study mentions two AI assistants: one to support MBSE workshops and another one to assist in modeling [7].

Longshore et al. introduced SysEngBench, a domain-specific multiple choice benchmark that evaluates LLMs on systems engineering questions across 10 topics reflecting the core processes of systems engineering [18].

Another study examined the use of LLMs to automatically establish a trace between system model artifacts and requirements that are managed in different IT systems [8]. Some studies demonstrated the feasibility of using LLMs for modeling system models in the textual notation of SysMLv2 [9,19]. Ghanawi et al. investigated the creation of SysML models from medical documents [10].

Most of these efforts primarily target the creation phase of system models, aiming to reduce entry barriers, while the usage phase of MBSE models remains largely overlooked [7–10,18–20]. This result is supported by Poulsen et al.'s investigation, which concluded that current research seems to focus on early SE activities, including modeling and analysis, such as requirement elicitation [21].

In a presentation published by the Systems Engineering Research Center, a chatbot system for querying ontologically backed digital threads using large language models was introduced. While the approach of combining knowledge graphs with LLM-based natural language processing appears promising for enabling conversational access to integrated product lifecycle data, it currently lacks completed scientific evaluation. The authors acknowledge that performance evaluation, including intent recognition accuracy, entity recognition accuracy, and user acceptance testing, is still ongoing. Furthermore, the system has yet to demonstrate robustness with complex analytical queries, and improvements to search accuracy, response time, and query processing for complicated questions remain listed as future work [22].

Pennock et al. [23] from The MITRE Corporation presented an approach for AI-enabled mission engineering at the AI4SE & SE4AI Research Application Workshop. Their research aims to augment large language models with RAG and GraphRAG techniques to enable mission engineers to interrogate complex SysML architecture models and AFSIM simulation models through natural language queries. While the approach addresses a genuine challenge, namely that current mission engineering takes longer than the time available for investment decisions, the work remains in an early stage. The authors acknowledge several significant limitations, including that commercial LLMs are not trained on defense-specific data, proprietary vendor data models make extraction difficult, sparse documentation in mission engineering models poses challenges for AI processing, and data sensitivity restricts both corpus size and available LLM options. Human subjects experiments to quantify actual time savings and quality improvements are planned but have not yet been conducted, with experimental use on actual mission engineering projects scheduled for the following year.

Bader et al. [11] present a proof-of-concept Graph Retrieval-Augmented Generation pipeline that treats SysML v2 models as semantic knowledge graphs, enabling the integration of model data into LLM prompts for natural language querying. While the approach demonstrates basic feasibility, it has notable limitations: the pipeline relies solely on semantic search with a single retrieval step, uses fixed-depth neighbor traversal (one-hop) without advanced graph exploration strategies, and has only been evaluated on a small illustrative SysMLv2 model.

As MBSE seeks to establish a consistent connection between engineering artifacts across the entire product lifecycle, it is crucial that heterogeneous stakeholder groups

from different disciplinary backgrounds can access and utilize this information. A major challenge in this context is that domain experts without MBSE training have limited MBSE expertise, making it difficult for them to understand the models and efficiently find the required information.

2.3. Chatbots and Knowledge Augmentation Techniques

Given the difficulty that non-MBSE-trained domain experts face when interacting with system models, tools that simplify access to model information are essential. One promising solution is the use of chatbots. Chatbots are increasingly applied across domains such as education, e-commerce, healthcare, and entertainment due to their ability to provide scalable, on-demand assistance independent of expert availability [24].

Transferring this concept to MBSE offers the potential to create interactive assistants capable of answering user queries about the system of interest. To achieve this, large language models that power the chatbot must be able to retrieve relevant information from system models.

2.3.1. Retrieval-Augmented Generation

With LLMs, this capability is enabled through Retrieval-Augmented Generation (RAG) [25], a rapidly evolving research area. In RAG applications, semantically relevant records are retrieved from a database and incorporated into the prompt for response generation. A RAG system consists of three main components: a query encoder, a retriever, and a generator. The query encoder transforms the user query into a vector representation. The retriever uses this representation to find relevant documents from a vector database. The generator then produces a response based on the query and the retrieved information [26].

Conventional RAG implementations store documents as text chunks in vector databases. Retrieval relies on embedding-based similarity search. This approach works well for fact-based queries answerable with a small subset of documents. However, it struggles with queries that require reasoning over relationships or global patterns across an entire dataset [27,28].

2.3.2. Graph Retrieval-Augmented Generation

Graph Retrieval-Augmented Generation (GraphRAG) addresses the limitations of conventional RAG. GraphRAG retrieves structured information from knowledge graphs instead of text corpora. It leverages explicit entity and relationship representations to enable more precise retrieval. The explicit representation of entities and relationships in graph databases aligns naturally with the structure of system models. System models consist of elements (nodes) connected by relationships (edges). This makes GraphRAG particularly promising in MBSE applications.

Peng et al. [29] decompose GraphRAG into three stages: Graph-Based Indexing (G-Indexing), Graph-Guided Retrieval (G-Retrieval), and Graph-Enhanced Generation (G-Generation). The main components for these three stages are shown in Table 1.

Graph-Based Indexing prepares the data for retrieval. The indexing method determines how the graph data is organized for retrieval. Graph indexing preserves the entire structure and enables graph search algorithms such as breadth-first search or shortest path [30]. Text indexing converts graph data into textual descriptions using templates or LLM-generated summaries [31]. Vector indexing transforms nodes and edges into vector embeddings for similarity-based search [32,33]. Hybrid indexing combines multiple methods to leverage their respective strengths [29,34].

Graph-Guided Retrieval extracts relevant information from the graph database. This stage involves decisions about the retriever type, retrieval paradigm, retrieval granularity, and query enhancement.

Table 1. GraphRAG framework showing design choices across three stages by Peng et al. [29].

Stage	Component	Options
G-Indexing	Indexing Method	Graph/Text/Vector/Hybrid
G-Retrieval	Retriever Type	Non-parametric/LM-based/GNN-based
	Retrieval Paradigm	Once/Iterative (Non-adaptive)/Iterative (Adaptive)/Multi-stage
	Retrieval Granularity	Nodes/Triplets/Paths/Subgraphs/Hybrid
	Query Enhancement	Query Expansion/Decomposition
G-Generation	Generator	GNN/LM/Hybrid
	Graph Format	Graph Language/Graph Embedding

The retriever type determines the underlying model for retrieval. Non-parametric retrievers use heuristic rules or graph algorithms and do not rely on deep-learning models, thereby achieving high efficiency [29]. An example for non-parametric retrievers is the retrieval of all entities along the shortest paths between two or more matched entities from a query in the knowledge graph [35]. LM-based retrievers leverage language models for natural language understanding and can generate reasoning paths or relevant relations. Jiang, Zhou et al. have developed a framework for LLMs to reason over structured data by augmenting the LLMs with a set of tools [36]. Graph neural network (GNN)-based retrievers encode graph structure and score candidates based on query similarity [29]. Many methods combine retriever types to balance efficiency and accuracy. For instance, GNN-RAG combines graph neural networks for reasoning over knowledge graph subgraphs with large language models [37].

The retrieval paradigm determines when and how often retrieval occurs. The retrieval gathers all information in a single operation using embedding similarities or pre-defined rules. For instance, HippoRAG is a single-step retrieval framework inspired by hippocampal indexing theory that orchestrates LLMs, knowledge graphs, and Personalized PageRank [38]. Another study explores single-step retrieval and reranking of relevant triplets from knowledge graphs, concatenating them with questions as input to language models [31]. Iterative retrieval conducts multiple searches where non-adaptive methods use fixed termination conditions and adaptive methods let models autonomously determine when to stop. An example for a non-adaptive method is PullNet, which uses a graph convolutional network to progressively construct question-specific subgraphs by repeatedly retrieving relevant information [39]. StructGPT introduces an iterative reading-then-reasoning approach where LLMs alternately collect relevant evidence from structured data and perform reasoning based on the gathered information [36]. KG-Agent proposes an autonomous agent framework that iteratively selects tools from a multifunctional toolbox and updates a knowledge memory to progressively reason over knowledge graphs [40]. Graph Chain-of-Thought augments LLMs with graph reasoning through iterative cycles consisting of LLM reasoning, LLMgraph interaction, and graph execution sub-steps [41]. The Observation-Driven Agent framework enhances KG reasoning through a cyclical paradigm of observation, action, and reflection, incorporating a recursive observation mechanism to manage the exponential growth of knowledge during exploration [42]. KnowledGPT employs program-of-thought prompting to generate executable

search queries for knowledge bases, enabling both retrieval and storage of knowledge through predefined functions [43]. Multi-stage retrieval divides the retrieval process into multiple stages and combines different approaches sequentially [29].

The retrieval granularity specifies what units are retrieved. Nodes represent individual entities. Triplets are subject–predicate–object tuples providing structured relational data. Paths capture sequences of relationships between entities and enable multi-hop reasoning. Subgraphs capture comprehensive relational contexts including ego graphs or k-hop neighborhoods. Hybrid approaches retrieve multiple granularities as needed based on the query [29].

For node-level retrieval, ATLANTIC constructs a heterogeneous document graph and retrieves passages using a graph neural network as a structural encoder [44]. GNN-Ret builds a graph connecting structure-related and keyword-related passages and applies a graph neural network to exploit inter-passage relationships [45]. HippoRAG orchestrates knowledge graphs with the Personalized PageRank algorithm to retrieve relevant entities [38]. For triplet retrieval, KG-Rank retrieves triples from a medical knowledge graph and applies ranking techniques to refine their ordering [46]. At path granularity, HyKGE defines three path types and employs corresponding rules to retrieve each from the knowledge graph [30]. At subgraph level, Peng and Yang retrieve ego graphs of patent phrases to augment embeddings with the global context [47]. Hybrid approaches include Graph-CoT, which iteratively reasons on text-attributed graphs through cycles of reasoning, interaction, and execution [41]; KG-Agent, an autonomous agent framework that selects tools and updates memory during reasoning [40]; KnowledGPT, which generates search operations in code format [43]; and ODA, which incorporates global observation through a recursive mechanism [42].

Query enhancement techniques improve retrieval quality through query expansion, which adds related terms or generates relation paths, and query decomposition, which breaks complex queries into smaller sub-queries [29].

Graph-Enhanced Generation produces the final response based on the query and retrieved graph data. The generator can be a GNN for discriminative tasks, an LM for text generation, or a hybrid combining both approaches. The graph format determines how graph data is presented to the generator. It can be divided into graph languages and graph embeddings. Graph languages represent graphs as text using adjacency tables, natural language, code-like forms, syntax trees, or node sequences. Graph embeddings encode graphs as vectors, e.g. by using GNNs. This allows handling long text inputs but leads to information losses of specific entity names [29].

In Section 3.3, this work is categorized into the GraphRAG framework by Peng et al. for comparison.

2.4. Research Need

The explicit representation of entities and their relationships in graph databases aligns naturally with the structure of MBSE system models, which are comparable to node–relationship constructs. This makes GraphRAG particularly promising for this use case.

Current SysMLv2 implementations, however, rely on a relational database with more than 8000 tables for model storage. While these ensure the preservation of complete syntactic information necessary for model exchange and tool interoperability, they introduce significant overhead when extracting semantically relevant content. As a result, system navigation and information retrieval remain cumbersome, limiting the practical integration of MBSE with LLM-based assistants. Tewari et al. present an approach that transfers model-based systems engineering (MBSE) models from SysML into a graph database to enable system analysis, such as determining which components fail during electrical

shorts or finding alternative operational routes when conditions are not met [48]. However, a key drawback of this methodology is that engineers must learn to write queries for the database to perform these analyses, which may limit adoption despite the powerful analytical capabilities the approach offers.

MBSE system models are curated sets of structured engineering data and knowledge and are highly valuable assets in system development. However, many stakeholders lack familiarity with modeling approaches, specialized modeling tools, or the modeling language itself, limiting their ability to read, interpret, and utilize this valuable information beyond the domain of systems engineers. Thus, there is a need to explore how this information can be made accessible to diverse stakeholders.

Large language models offer promising potential to address this challenge, as they enable natural language interaction and can effectively navigate complex data structures through techniques such as Graph Retrieval-Augmented Generation. Knowledge graphs provide a suitable representation for storing and querying the structured information contained in MBSE system models.

This paper addresses this challenge by proposing an LLM-based multi-agent system with an individualized GraphRAG strategy for retrieving system model information. We demonstrate the benefit of this approach with a chatbot that answers questions about underlying system models. This enables both stakeholders and AI systems to access MBSE data, laying the groundwork for future MBSE-enhanced AI applications in the engineering domain.

To achieve this objective, the following research questions are investigated:

1. How can MBSE system models that follow an RFLP modeling approach be transferred into a knowledge graph?
2. How can a retrieval strategy be designed that leverages the metamodel of the system model?

3. Materials and Methods

The proposed methodology is organized into four phases. In the first phase, a preprocessing pipeline is developed that converts system models from the textual notation of SysMLv2 into a knowledge graph. This involves translating the metamodel into a corresponding graph schema. Building on this knowledge graph, the second phase implements a hierarchical multi-agent system that leverages GraphRAG with a customized retrieval strategy to answer user queries. In the third phase, to demonstrate and evaluate the approach, a reference architecture of a battery electric vehicle is developed using SysMLv2 textual notation. The model covers all four levels of abstraction in the RFLP methodology, ensuring full traceability from requirements to functional decomposition and logical and physical implementation. Finally, the fourth phase evaluates the approach using a Q/A dataset comprising 100 questions, to assess its performance and reliability. Figure 1 shows the preprocessing and user interaction part of our methodology.

3.1. Preprocessing Pipeline

A key distinction between classical GraphRAG applications and the proposed MBSE use case lies in the nature of the underlying data. Traditional GraphRAG implementations typically operate on unstructured or semi-structured sources such as text corpora like scientific articles, where the graph structure must first be inferred through entity extraction and relation identification. In contrast, MBSE system models already provide inherently structured data created by human experts, consisting of explicitly defined entities and relationships. This eliminates the need for entity extraction and entity-linking steps, allowing the graph representation to directly reflect the system architecture.

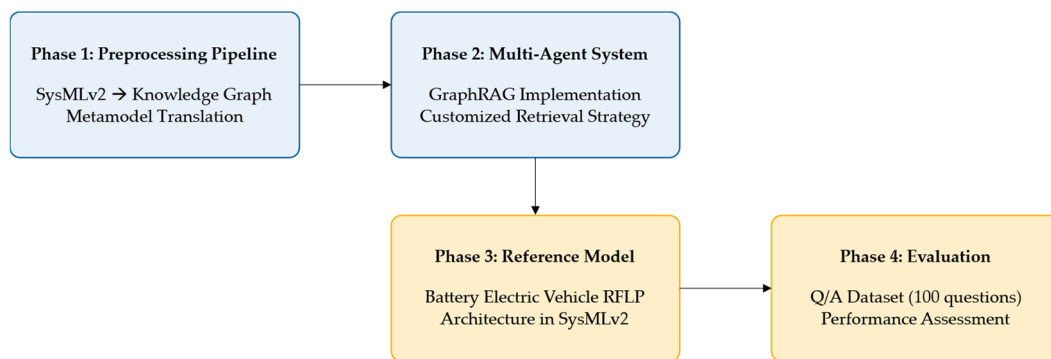


Figure 1. Overview of the four-phase methodology: development of the preprocessing pipeline and multi-agent system (Phases 1–2), followed by application to a BEV reference model and evaluation (Phases 3–4).

The overarching methodology is designed to be adaptable to any SysMLv2 model by developing a customized parser for the specific metamodel structure. However, the parser implementation developed for this work is specifically tailored to the RFLP (Requirements, Functions, Logical, Physical) abstraction framework and the particular SysMLv2 textual notation syntax used in this modeling approach. The parser cannot be directly applied to other SysMLv2 modeling methodologies without adaptation, as different metamodels may define different entity types, relationships, and syntactic structures.

The first step of the preprocessing pipeline involves constructing a graph schema grounded in the chosen modeling methodology. This is achieved by mapping elements from the SysMLv2 textual notation to corresponding nodes and relationships in the graph. The metamodel defines the syntactic structure of the SysMLv2 notation and specifies which entity types and relationships are permissible within the system model.

In the second step, a custom Python-based (Python 3.12) parser is developed to process the SysMLv2 textual notation. The parser uses regular expressions and pattern matching to identify and extract entity definitions and their usages from the SysMLv2 code. It generates intermediate data structures in Python that represent the system model's entities (requirements, functions, logical elements, physical elements, ports) and their relationships (satisfies, performs, is_composed_of, flows_to, is_connected_to, is_allocated_to). These intermediate structures are then converted into Cypher, the query language of the Neo4j graph database, to populate it. Cypher enables efficient querying and traversal of graph structures through pattern-matching syntax. To minimize graph complexity, only nodes representing instances of SysMLv2 entities are created, while entity definitions are omitted from the graph representation.

The parser distinguishes between entity definitions and entity usages in the SysMLv2 code. Definitions specify the structure and properties of reusable entity types (e.g., requirement definitions, function definitions), while usages instantiate these definitions with specific names and parameters. The parser extracts key information from both definitions and usages, such as descriptions (from documentation comments), parameters (inputs/outputs for functions, attributes for physical elements), and relationships (satisfied requirements, hierarchical decompositions, interface connections).

Figure 2 illustrates the resulting graph schema for the RFLP modeling approach. At the requirement level (indicated in red), the system model comprises functional requirements (FR), performance requirements (PR), resource requirements (RR), and design requirements (DR). At the functional layer (yellow), functions (F) are defined, each associated with function inputs (FI) and function outputs (FO). Function inputs and function outputs are linked via the flows_to relationship, while functions themselves can be hierarchically

decomposed through the *is_composed_of* relationship. Additionally, functions are linked to functional requirements using the *must_satisfy* relationship.

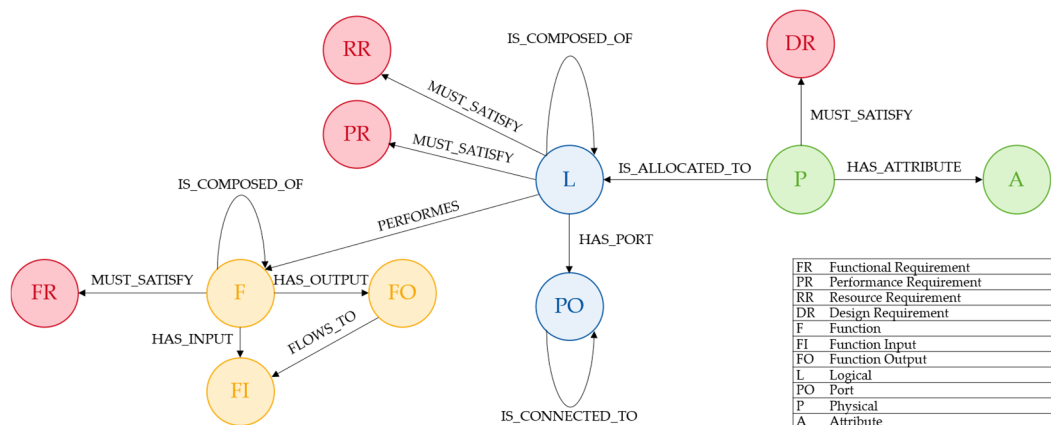


Figure 2. Graph schema used for the RFLP modeling approach. Color legend: red = requirements, yellow = functions, blue = logical elements, green = physical elements.

The logical layer, indicated in blue, consists of logical elements (L) and ports (PO). Logical elements can be hierarchically decomposed via *is_composed_of* and perform functions as represented by the *performs* relationship. Logical elements are also connected to performance and resource requirements through the *must_satisfy* relationship. Ports are interconnected via the *is_connected_to* relationship, enabling the exchange of material, energy, or information between logical elements.

At the physical layer (green), physical elements (P) are associated with attributes (A) through the *has_attribute* relationship and are required to satisfy design requirements via the *must_satisfy* relationship.

This preprocessing methodology transforms the system model from SysMLv2 code into a fully connected knowledge graph that preserves the semantics and traceability across the RFLP abstraction levels.

3.2. Multi-Agent System and Retrieval Strategy

The chatbot and its retrieval strategy are implemented through two cooperating agents, referred to as the Supervisor Agent (SA) and the Graph Query Agent (GQA). The SA is responsible for managing the interaction with the user, maintaining the chat history, interpreting the user's intent, decomposing queries into subtasks, planning the retrieval process, and reacting to intermediate or final results. To support this process, the supervisor employs a semantic search tool that relies on cosine similarity to identify semantically relevant nodes within the knowledge graph. Cosine similarity is a metric that measures the similarity between two vectors by calculating the cosine of the angle between them, with values close to 1 indicating high similarity and values close to 0 indicating low similarity. The search is performed on vector embeddings of the concatenation of every node's name and description in the knowledge graph. Vector embeddings are high-dimensional numerical representations of text that capture semantic meaning, enabling mathematical comparison of textual content. The search input to this tool can be various natural language forms, ranging from single words to complete sentences.

Figure 3 shows the process that guides the supervisor's actions. This process is defined in the system prompt. It begins with identifying the type of question posed by the user. The classification comprises two categories. The first category includes questions that require the identification of one or more specific entities, where a name or description of the entity is provided, such as identifying the design requirements for the e-motor. In this case, the

semantic search is used to resolve the precise name and type of the node. This eliminates the need for a user to know the exact name of an element when asking a question. The second category encompasses questions that do not target specific entities but request more general information, such as listing all physical components of a system.

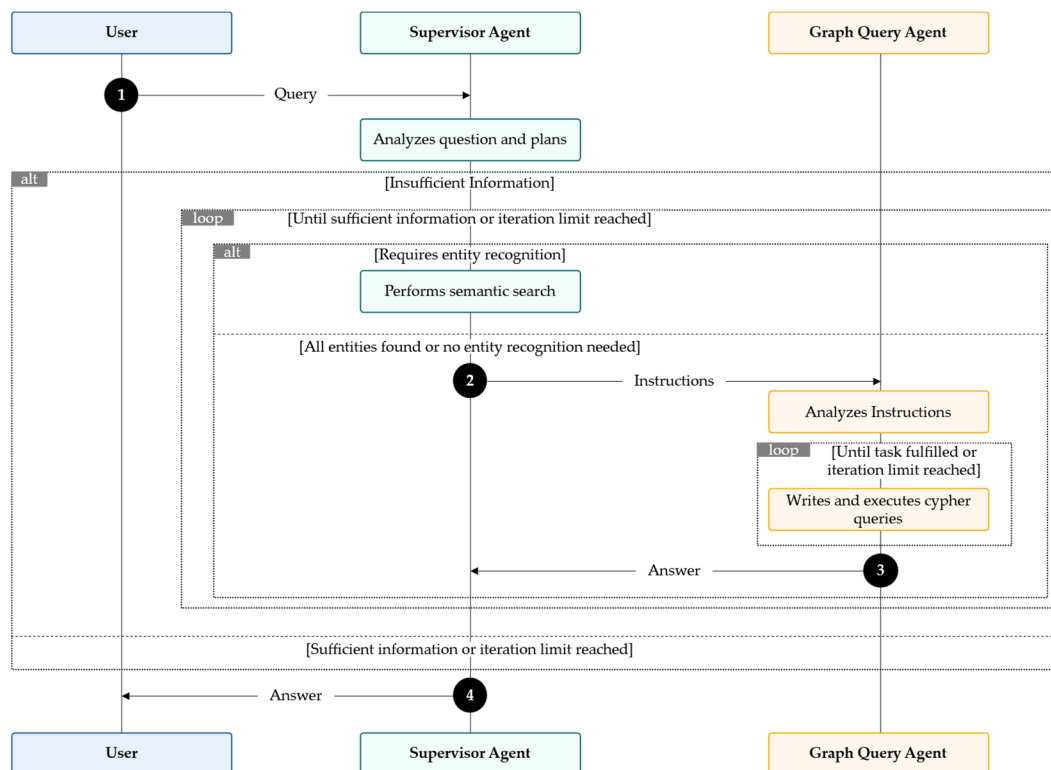


Figure 3. Procedure of the multi-agent system from a user query to a final response.

In both categories, the supervisor delegates the final retrieval step to the Graph Query Agent, ensuring that all retrieved information is traceable through a structured query. This allows every step of the query process to be mapped directly to the corresponding database result. When invoking the GQA, the supervisor provides an instruction that describes the intended query. This instruction is supplemented by a static prompt containing few-shot examples of valid Cypher queries aligned with the graph schema, which serves as guidance for query formulation.

The GQA processes the supervisor's instructions and iteratively generates Cypher queries until the retrieval task is successfully completed. The results are then returned to the supervisor, who evaluates whether the retrieved information is sufficient to answer the user's request. Both agents operate according to the Reason-Act (ReAct) pattern, reasoning over the intent and observations before taking the corresponding action [49].

3.3. Categorization Within GraphRAG Framework

Table 2 positions this work within the GraphRAG framework by Peng et al. presented in Section 2.3.2.

This work uses a hybrid indexing approach that combines graph and vector indices. Neo4j provides native graph indexing, which preserves the structure of the system model and enables traversal algorithms. An additional vector index on nodes enables searches based on semantic similarity. The retriever is LM-based, as natural language queries must first be translated into Cypher queries. Neo4j supports the execution of search algorithms on the graph. In theory, this allows agents to identify the shortest paths between two entities by composing a simple Cypher query. Language models interpret user intent and generate

the appropriate database queries. GNN-based retrievers offer an alternative approach or extension to this method that could be explored in future work. The retrieval paradigm is iterative and adaptive. Complex questions about system models may require multiple database queries to gather sufficient information. The agents autonomously decide when they have retrieved enough data to answer the question.

Table 2. Design choices of this work within the GraphRAG framework by Peng et al.

Component	Options	This Work
Indexing Method	Graph/Text/Vector/Hybrid	Hybrid
Retriever Type	Non-Parametric/LM-Based/GNN-Based	LM-based (Multi-Agent System)
Retrieval Paradigm	Once/Iterative (Non-Adaptive)/Iterative (Adaptive)/Multi-Stage	Iterative, Adaptive (ReAct pattern)
Retrieval Granularity	Nodes/Triplets/Paths/Subgraphs/Hybrid	Hybrid (All Granularities via Cypher)
Query Enhancement	Query Expansion/Decomposition	Query Decomposition (Supervisor Agent)
Generator	GNN/LM/Hybrid	LM (Same Model as Retriever)
Graph Format	Graph Language/Graph Embedding	Graph Language, Code-Like (Cypher, JSON)

The retrieval granularity is hybrid because different questions require different retrieval units. Some questions need single nodes while others require paths or entire subgraphs. Cypher queries can retrieve any granularity as needed. The Supervisor Agent performs query decomposition by breaking complex questions into smaller sub-queries. Each sub-query is handled separately before combining the results into a final response. The generator is an LM because the chatbot requires natural language responses. The graph format is code-like using Cypher queries and JSON results. Cypher provides precise and unambiguous graph queries that Neo4j executes natively.

3.4. Reference Model: Battery Electric Vehicle Architecture

To demonstrate and evaluate the proposed methodology, a reference architecture of a battery electric vehicle (BEV) was developed using SysMLv2 textual notation. Creating a comprehensive system model manually would be prohibitively time-consuming; therefore, the LLM Claude Sonnet 3.7 and the LLM-powered search engine Perplexity were employed to accelerate the model generation process. Perplexity was used to collect information on the requirements, functions, logical and physical components of a battery electric vehicle. Independently, the author defined a metamodel specifying the SysMLv2 syntax to follow. Claude was then provided with this metamodel along with the RFLP information gathered by Perplexity and prompted to generate a SysMLv2 model of the battery electric vehicle in accordance with the metamodel guidelines. The resulting model was subsequently manually verified for syntactic correctness. It is important to note that the use of AI-assisted generation is not a prerequisite of the methodology; the preprocessing pipeline and multi-agent system work with any valid SysMLv2 model following the syntax of the metamodel.

The completed system architecture shown in Figure 4 comprises 97 requirements, 37 functions, 9 logical components, and 34 physical components. The resulting knowledge graph contains 427 nodes and 505 relationships, representing a fully connected, multi-layered system model.

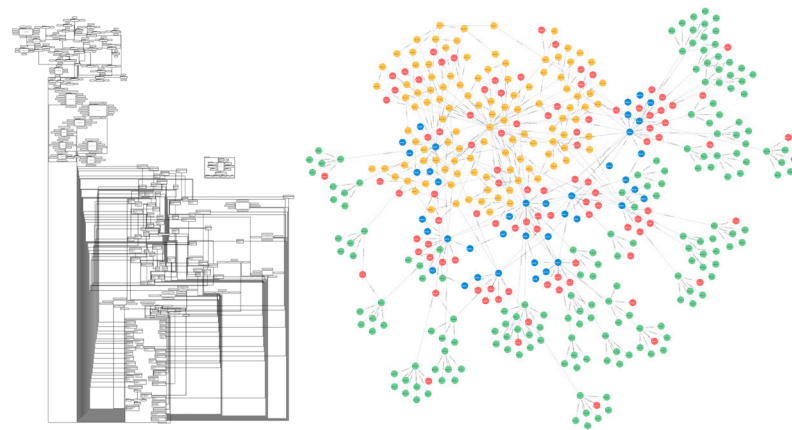


Figure 4. Left: system architecture visualized with TomSawyer. Right: instances of the system architecture represented as a knowledge graph in neo4j. Color legend: red = requirements, yellow = functions, blue = logical elements, green = physical elements.

3.4.1. Requirements Architecture

The requirements architecture establishes four distinct categories. Functional requirements (37 total) define capabilities that the system must provide, including user authentication, real-time diagnostics, regenerative braking, and advanced driver assistance features. Performance requirements (28 total) specify quantitative measures such as driving range (400 km minimum), acceleration performance (0–100 km/h in <8 s), and system response times. Resource requirements (16 total) constrain system utilization including energy consumption (<16 kWh/100 km), power draw limits for subsystems, and environmental operating conditions. Design requirements (16 total) establish physical constraints including mass limits (motor < 80 kg), dimensional envelopes, and integration constraints.

3.4.2. Functional Architecture

The functional architecture decomposes vehicle operations hierarchically under a top-level system coordination function. Functions span different domains: overall vehicle operations (authentication, diagnostics, connectivity), propulsion control (power delivery, drive modes, regenerative braking), energy management (cell monitoring, state estimation, thermal control), thermal regulation (active cooling, HVAC integration), vehicle dynamics (stability monitoring, intervention control), braking systems (dual-mode control, ABS/EBD), and driver assistance (object detection, lane keeping, emergency braking). Each function defines specific input/output flows using domain-specific signal types including data signals, control commands, and various energy forms like electrical, mechanical, and thermal energy.

3.4.3. Logical Architecture

The logical architecture is divided into interconnected subsystems, each of which satisfies performance and resource requirements. Additionally, logical elements perform functions. The logical element “OverallVehicleLogical” serves as the top-level coordinator, integrating specialized logical elements:

- “ConnectivityLogical” for user interfaces and connectivity;
- “PropulsionSystemLogical” for motor control and power conversion;
- “EnergyStorageLogical” for battery management and cell monitoring;
- “ThermalManagementLogical” for cooling and heating control;
- “VehicleControlLogical” for vehicle control;
- “StabilityControlLogical” for dynamics monitoring and intervention;

- “BrakingSystemLogical” for dual-mode braking and energy recovery;
- “ADASSystemLogical” for sensor fusion and automated functions.

Interfaces are used to exchange material, energy, or information between logical elements using ports (31 total).

3.4.4. Physical Architecture

The physical architecture realizes logical components through distinct hardware elements spanning electronic control units, sensors, actuators, and mechanical components. Among others, they include a permanent magnet synchronous motor with attributes like a peak power of 150 kW and a maximum torque of 350 Nm, a 100 kWh NMC811 battery pack, an ASIL-D-compliant battery management unit, automotive-grade processors (Infineon AURIX TC397 for vehicle control, NVIDIA Orin for ADAS processing), and comprehensive sensor suites including cameras, radar, lidar, and inertial measurement units.

3.5. Question-and-Answer Dataset

To evaluate the proposed approach, a question–answer dataset was created. The dataset is structured into four categories (see Table 3). Questions are divided according to the number of relationships that must be traversed within the knowledge graph to gather all required information. A zero-to-one-hop question requires information that can be obtained either from a single node or from two nodes directly connected by a single relationship. In contrast, multi-hop questions demand the traversal of two or more relationships to retrieve the correct answer. By design, zero-to-one-hop questions are expected to be less complex and thus easier to resolve than multi-hop questions. The final dataset comprises 100 questions in total: 100 answerable questions, evenly split into 50 zero-to-one-hop and 50 multi-hop questions.

Table 3. Q/A categories in the evaluation dataset.

Subcategory	Quantity
Zero-to-one-hop	50
Multi-hop	50

4. Results

This study presents a methodology that enables AI-based interaction with MBSE system models created in SysMLv2. Figure 5 illustrates the final architecture. The architecture is composed of the SysMLv2 layer, a graph layer, and the multi-agent system.

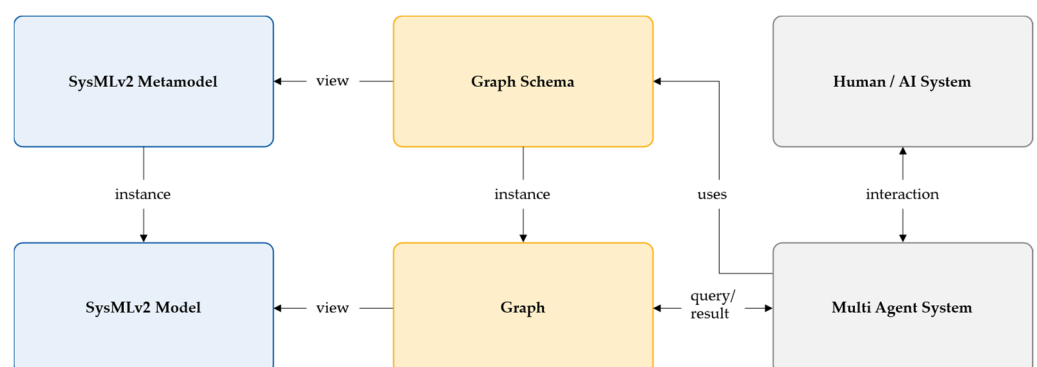


Figure 5. Architecture for an LLM-based multi-agent system using a GraphRAG retrieval strategy to access system model information.

4.1. Architecture

The SysMLv2 metamodel defines the structural and semantic rules for system modeling. A SysMLv2 model is instantiated from this metamodel. To enable graph-based processing, the metamodel is transformed into a graph schema, which can be interpreted as a view on the metamodel. The SysMLv2 model is subsequently transformed into a graph structure that conforms to this graph schema, thereby representing a view on the SysMLv2 model.

The graph schema is used to instruct the agents of the multi-agent system through their prompts. By querying the graph, the system retrieves relevant information and generates responses to questions about the system. Users and other AI systems interact with the multi-agent system through natural language, which processes their requests by accessing the structured graph representation.

The methodology is designed to be adaptable to different SysMLv2 modeling approaches, requiring only the definition of an appropriate graph schema and parser implementation for the specific metamodel in use.

4.2. Evaluation

To evaluate this approach, the methodology was applied to a battery electric vehicle reference architecture and tested with a question–answer dataset. The following section presents the evaluation results.

We evaluated the performance of four large language models, namely Llama-3.3-70B-Instruct-Turbo-Free [50], Gemini-2.0-flash-lite, Gemini-2.0-flash, and Gemini-2.5-flash-preview-04-17 [51]. We have selected these LLMs due to their open-source availability. In all experimental settings, the same model was employed simultaneously as the Supervisor Agent and as the Graph Query Agent, ensuring that performance differences can be attributed directly to the model rather than to heterogeneous agent configurations. The evaluation considered the previously introduced categories of questions, which distinguish between zero-to-one-hop and multi-hop reasoning.

The results are presented in Table 4. For zero-to-one-hop questions, all models achieved high accuracy, with Gemini-2.5-flash-preview-04-17 performing best at 96%. In contrast, multi-hop questions proved more challenging. Here, performance varied substantially, ranging from 50% for Llama-3.3-70B-Instruct-Turbo-Free to 90% for Gemini-2.5-flash-preview-04-17. The evaluation of related questions demonstrated particularly strong results for Gemini-2.5-flash-preview-04-17, which achieved 100% accuracy, meaning it clearly stated in all cases that the information is not present in the knowledge graph.

Table 4. Quantity of correct answers and accuracy of the LLM-based multi-agent system.

Question Category	Gemini 2.5 Flash	Gemini 2.0 Flash	Gemini 2.0 Flash Lite	Llama-3.3-70B-Instruct-Turbo
One-hop	48 (96%)	47 (94%)	44 (88%)	47 (94%)
Multi-hop	45 (90%)	41 (82%)	32 (64%)	25 (50%)
Average	93 (93%)	88 (88%)	76 (76%)	62 (62%)

When averaging across the answerable categories, Gemini-2.5-flash-preview-04-17 outperformed the other models with 93% accuracy, followed by Gemini-2.0-flash and Gemini-2.0-flash-lite at 88% and 76%, respectively. Llama-3.3-70B-Instruct-Turbo-Free reached 75% on average, indicating lower robustness across question types. Overall, the evaluation confirms that Gemini 2.5 Flash consistently achieved superior retrieval performance across both simple and complex query types.

Table 5 presents the average response times of the evaluated LLMs across the three question categories. The Gemini series was accessed via Google AI Studio API, while the LLaMA-3.3-70B-Instruct-Turbo model was accessed through Together.ai API. The differences in the average response time between the Gemini and Llama models are potentially caused by the API. Across all models, response times remained within a range that can be considered fast and suitable for interactive Q/A tasks. Among the configurations, Gemini 2.0 Flash Lite yielded the lowest average response time (6.15 s), while LLaMA-3.3 showed the longest latency (40.02 s). Although these differences exist, the primary finding is that the multi-agent system is able to generate answers in an acceptable response time across all evaluated settings. This underlines the feasibility of deploying such systems in an industrial use case.

In the following, we present an example of an answering process to a multi-hop question: Figure 6 describes how the multi-agent system collaboratively answers the user's technical question.

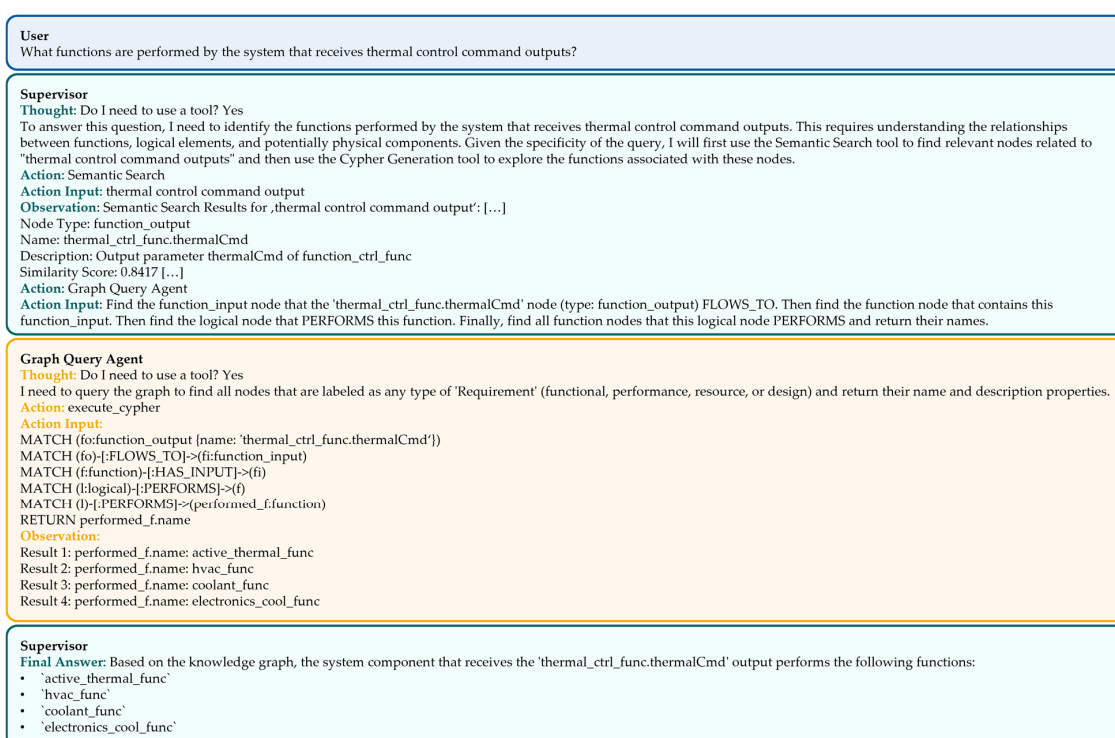


Figure 6. Example interaction flow from a user question to the final answer in a multi-hop question.

The user asks which functions are performed by the system that receives thermal control command outputs. The Supervisor Agent first reasons about the task, deciding it needs to locate the relevant function output in the knowledge graph. Since the exact node name of the function output is unclear, it invokes a Semantic Search Agent, which returns several candidate nodes related to thermal control. The most relevant match is identified as thermal_ctrl_func.thermalCmd. Next, the supervisor hands the query to a Graph Query Agent, which traces the graph structure. It follows the output node to the corresponding input node, identifies the function receiving this input, and then determines the logical element responsible for performing it. Finally, the agent collects all functions associated with this logical element. The result shows that the system receiving the thermal control command output performs the following functions: active_thermal_func, hvac_func, coolant_func, and electronics_cool_func.

thermal control example, the system follows edges from function outputs to receiving functions, and from logical elements to their associated functions, reconstructing the complete information flow across architectural layers. In the constraint violation case, the system navigates across multiple relationships of physical components to compare actual values against design limits.

Second, the graph structure supports analytical queries that require aggregation and comparison across the entire model. Identifying redundant requirements, for instance, involves systematically examining all requirement nodes and their descriptions, a task that benefits from the graph's ability to enumerate and compare entities of the same type. Detecting constraint violations requires joining information from multiple node types, which the graph query languages handle natively by traversing the relevant relationships and returning the data in a structured format that the LLM can then analyze.

These capabilities—relationship traversal, cross-entity comparison, and structured aggregation—are inherent to graph-based retrieval and enable the system to answer questions that go beyond simple fact lookup and would otherwise require substantial manual effort to evaluate across large-scale system models. The evaluation results confirm that the proposed multi-agent architecture achieves both high accuracy and acceptable response times, validating the methodology as a promising foundation for AI-assisted access to MBSE system models.

Table 5. Average response time of the LLM-based multi-agent system. Gemini series was utilized with Google AI Studio API and Llama was utilized with Together.AI API.

Question Category	Gemini 2.5 Flash [s]	Gemini 2.0 Flash [s]	Gemini 2.0 Flash Lite [s]	Llama-3.3-70B-Instruct-Turbo [s]
One-hop	10.03	5.79	5.23	28.96
Multi-hop	14.32	8.21	6.62	40.78
Average	11.29	8.54	6.15	40.02

5. Discussion

This section examines the evaluation results and discusses factors that influence the performance and applicability of the proposed approach.

5.1. Influence of Language Model Selection

The evaluation revealed substantial performance differences between the tested LLMs, particularly in multi-hop reasoning tasks. Gemini 2.5 Flash achieved 90% accuracy on multi-hop questions, while Llama-3.3-70B reached only 50%. This 40-percentage-point gap suggests that model size and training strongly influence the ability to maintain reasoning consistency across multiple retrieval steps. The superior performance of Gemini 2.5 Flash may be attributed to better instruction-following capabilities and more robust reasoning patterns. Furthermore, the system prompt and few-shot examples were not specifically optimized for each model, potentially disadvantaging some configurations. A more controlled comparison would require model-specific prompt engineering.

5.2. Response Time Considerations

The observed response times varied considerably, ranging from 6.15 s (Gemini 2.0 Flash Lite) to 40.02 s (Llama-3.3-70B). While all times remain within acceptable ranges for interactive use, these differences are influenced by multiple factors beyond model inference speed. API latency, network conditions, and server load contribute to the measured times, making direct performance comparisons between models difficult. The multi-agent

architecture itself introduces overhead through multiple LLM calls per query. Optimizing the agent interaction pattern, such as minimizing unnecessary reasoning steps, could improve response times across all configurations.

5.3. Impact of Model Characteristics on Accuracy

The choice of the reference system model influences evaluation outcomes in several ways. The battery electric vehicle architecture contains 427 nodes and 505 relationships, representing a moderately complex system. Larger industrial models with thousands of nodes and dense interconnections may challenge the retrieval strategy differently. The complexity of the graph probably has an impact on both the precision of semantic searches and the difficulty of formulating queries. The more nodes there are, the more likely it is that there will be an ambiguous match. Additionally, longer path traversals require more sophisticated Cypher queries.

The RFLP modeling approach provides a relatively clean hierarchical structure with well-defined relationship types. Models with more heterogeneous relationship semantics or less consistent naming conventions might degrade performance. Additionally, the question dataset was manually crafted to align with the model's content and structure. Real-world user queries may be more ambiguous, use domain-specific terminology, or reference information not explicitly modeled, which would likely reduce accuracy below the reported 93%.

5.4. Integration into MBSE Development Practice

For practical adoption, the proposed approach must be seamlessly integrated into existing MBSE workflows. Currently, the methodology requires exporting SysMLv2 models and processing them through a custom parser, which introduces friction. Native support for graph database export in MBSE tools would streamline this process. The approach could serve multiple roles in systems engineering practice. During requirements engineering, stakeholders could query requirement satisfaction across system levels without navigating complex model diagrams. During design reviews, engineers could quickly identify which physical components realize specific functions or verify traceability from requirements to implementation. For documentation, the system could automatically generate stakeholder-specific views of the architecture by querying relevant subgraphs. However, these use cases require user studies to validate that non-expert stakeholders can effectively interact with the system and trust its responses.

Beyond supporting non-expert stakeholders, the proposed approach could offer significant value for model developers themselves. During the development of large-scale system models, engineers could use the chatbot to quickly assess the current state of the model, such as identifying incomplete traceability links or verifying the coverage of requirements across abstraction levels. When new team members join an ongoing modeling effort, the system could provide an accessible entry point to explore the existing architecture without requiring extensive tool-specific training. Furthermore, for those responsible for maintaining and evolving the model throughout the system lifecycle, the natural language interface could enable efficient navigation and validation of complex model structures that would otherwise require significant manual effort.

5.5. Limitations and Scope

Several limitations constrain the generalizability of these findings. The custom parser implementation is specific to the RFLP metamodel and SysMLv2 textual notation used in this study. While the methodology is designed to be adaptable, applying it to other modeling approaches requires developing new parsers aligned with different metamodel structures. The evaluation was conducted on a synthetic architecture that, while reasonably

complex, does not capture the full heterogeneity and scale of industrial system models. Real-world models often contain incomplete or inconsistent information, which was not reflected in the controlled reference model.

An additional limitation concerns the quality and consistency of the underlying system models. The reference architecture used in this evaluation was deliberately constructed with consistent naming conventions, complete traceability links, and well-defined relationships. In practice, however, MBSE models often exhibit inconsistencies resulting from iterative development, multiple contributors, or evolving requirements. Missing relationships, ambiguous naming, incomplete documentation, and structural irregularities are common in real-world modeling efforts. The proposed approach has been evaluated using a well-formed model structure. Future work should investigate the robustness of the methodology when applied to models with varying degrees of completeness and structural quality.

Furthermore, the absence of user studies limits conclusions about usability and stakeholder acceptance. High accuracy on predefined questions does not guarantee that the system meets the actual information needs of systems engineers, project managers, or other stakeholders in practice. The lack of comparable approaches and standardized benchmarks for question-answering over MBSE models also hinders cross-study validation. The development of community-driven evaluation datasets, similar to efforts in other domains, would support more rigorous comparative analysis across different methodologies.

6. Summary and Outlook

This work presents a methodology for enabling AI-based natural language interaction with MBSE system models through knowledge graph representation and multi-agent retrieval. The approach addresses the challenge of making MBSE system models accessible to both AI systems and stakeholders without specialized modeling expertise. The methodology comprises a preprocessing pipeline that transforms SysMLv2 models into knowledge graphs based on a derived graph schema, and a hierarchical multi-agent system that leverages a customized retrieval strategy. Evaluation on a battery electric vehicle reference architecture with 100 questions demonstrated that the best-performing configuration (Gemini 2.5 Flash) achieved 93% accuracy with response times suitable for interactive use. The results indicate that AI systems can effectively retrieve structured information from MBSE system models, supporting the hypothesis that graph-based representations enable robust question-answering capabilities across different levels of query complexity.

Several directions for future work emerge from this study. First, evaluating the approach on larger-scale industrial MBSE system models is necessary to assess scalability and identify performance bottlenecks. Ongoing research in collaboration with automotive industry partners is examining these aspects. Second, testing different context engineering approaches, alternative GraphRAG retrieval strategies, and different agent architectures could improve performance and efficiency. This includes experimenting with different prompt structures, few-shot example selections, and agent coordination patterns to optimize both accuracy and response time. Third, an investigation of the advantages and disadvantages of alternative graph representations, such as the Resource Description Framework, with respect to accuracy, response time, and interoperability should be conducted. Finally, establishing standardized benchmarks for question-answering over MBSE system models would enable systematic comparison of different retrieval strategies and support community-driven improvements. Such benchmarks could include diverse MBSE system models and query complexities. The development of native graph database support in MBSE tools represents a particularly promising direction, as it would eliminate the need for external preprocessing and enable real-time synchronization between model edits and graph representation. This would result in MBSE models becoming knowledge sources

that can be queried directly. This fundamental change would impact how stakeholders interact with system architectures throughout the development lifecycle.

Author Contributions: Conceptualization, V.Q.; methodology, V.Q.; software, V.Q.; validation, V.Q.; formal analysis, V.Q.; investigation, V.Q.; resources, V.Q.; data curation, V.Q.; writing—original draft preparation, V.Q.; writing—review and editing, V.Q., G.J., G.H. and S.D.; visualization, V.Q.; supervision, S.D. and G.H.; project administration, S.D.; funding acquisition, S.D. All authors have read and agreed to the published version of the manuscript.

Funding: This research was conducted in the context of the publicly funded research project “KIMBA—Künstliche Intelligenz für Systemmodellbildung und Anforderungen” by the Federal Ministry of Economic Affairs and Energy with the grant number FKZ: 19S24001I. The APC was funded by RWTH Aachen University Open Access Publication Fund.

Data Availability Statement: The data presented in this study are available on request from the corresponding author.

Acknowledgments: During the preparation of this manuscript/study, the authors used Perplexity and Claude for the purposes of creating an artificial MBSE system architecture in SysMLv2. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The funders had no role in the design of the study; in the collection, analysis, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

AI	Artificial Intelligence
GenAI	Generative Artificial Intelligence
GNN	Graph Neural Network
GraphRAG	Graph Retrieval-Augmented Generation
LM	Language Model
LLM	Large Language Model
MBSE	Mode-Based Systems Engineering
RAG	Retrieval-Augmented Generation
SE	Systems Engineering

References

- Salado, A. Introduction to Systems Engineering in the 21st Century: Volume 1. In *Systems Engineering Series*; Cuadernos de Isdefe: Madrid, Spain, 2024; ISBN 978-84-695-7816-2.
- Griffor, E.R.; Greer, C.; Wollman, D.A.; Burns, M.J. *Framework for Cyber-Physical Systems: Volume 1, Overview*; National Institute of Standards and Technology (NIST): Gaithersburg, MD, USA, 2017; Volume 1.
- Madni, A.M.; Sievers, M. Model-based systems engineering: Motivation, current status, and research opportunities. *Syst. Eng.* **2018**, *21*, 172–190. [CrossRef]
- Henderson, K.; Salado, A. Value and benefits of model-based systems engineering (MBSE): Evidence from the literature. *Syst. Eng.* **2021**, *24*, 51–66. [CrossRef]
- Chami, M.; Bruel, J.-M. A Survey on MBSE Adoption Challenges. In Proceedings of the INCOSE EMEA Sector Systems Engineering Conference (INCOSE EMEASEC 2018), Berlin, Germany, 8 November 2018; pp. 1–16. Available online: <https://hal.science/hal-02124402v1/document> (accessed on 8 January 2026).
- Förster, F.; Koldewey, C.; Bernijazov, R.; Dumitrescu, R.; Bursac, N. Navigating viewpoints in MBSE: Challenges, potential and pathways for stakeholder participation in industry. *Proc. Des. Soc.* **2025**, *5*, 2531–2540. [CrossRef]
- Bernijazov, R.; Dumitrescu, R.; Hanke, F.; von Heißen, O.; Kaiser, L.; Tissen, D. AI-Augmented Model-Based Systems Engineering. *Z. Für Wirtsch. Fabr.* **2025**, *120*, 96–100. [CrossRef]
- Bonner, M.; Zeller, M.; Schulz, G.; Savu, A. LLM-based Approach to Automatically Establish Traceability between Requirements and MBSE. *INCOSE Int. Symp.* **2024**, *34*, 2542–2560. [CrossRef]

9. DeHart, J.K. Leveraging Large Language Models for Direct Interaction with SysML v2. *INCOSE Int. Symp.* **2024**, *34*, 2168–2185. [CrossRef]
10. Ghanawi, I.; Chami, M.W.; Chami, M.; Coric, M.; Abdoun, N. Integrating AI with MBSE for Data Extraction from Medical Standards. *INCOSE Int. Symp.* **2024**, *34*, 1354–1366. [CrossRef]
11. Elias, B.; Jounes, A.C.; Katharina, P.; Markus, M.P.; Christian, N. Integrating SysML v2 into a GRAG LLM Pipeline: Design, Implementation and Evaluation. ResearchGate. 2025. Available online: https://www.researchgate.net/publication/395749011_Integrating_SysML_v2_into_a_GRAG_LLM_Pipeline_Design_Implementation_and_Evaluation (accessed on 8 January 2026). [CrossRef]
12. INCOSE. MBSE-Initiative. Available online: <https://www.incose.org/communities/working-groups-initiatives/mbse-initiative> (accessed on 26 August 2025).
13. OMG. SysML v2: The Next Generation Systems Modeling Language. Available online: <https://www.omg.org/sysml/sysmlv2/> (accessed on 18 November 2025).
14. Estefan, J.A. Survey of model-based systems engineering (MBSE) methodologies. *IncoSE MBSE Focus Group* **2007**, *25*, 1–12.
15. Bonnet, S.; Voirin, J.-L.; Normand, V.; Exertier, D. Implementing the MBSE Cultural Change: Organization, Coaching and Lessons Learned. *INCOSE Int. Symp.* **2015**, *25*, 508–523. [CrossRef]
16. Bayer, T.; Day, J.; Dodd, E.; Jones-Wilson, L.; Rivera, A.; Shougarian, N.; Susca, S.; Wagner, D. Europa Clipper: MBSE Proving Ground. In Proceedings of the 2021 IEEE Aerospace Conference (50100), Big Sky, MT, USA, 6–13 March 2021; IEEE: New York, NY, USA, 2021; pp. 1–19, ISBN 978-1-7281-7436-5.
17. Call, D.R.; Herber, D.R.; Conrad, S.A. The Effects of the Assessed Perceptions of MBSE on Adoption. *INCOSE Int. Symp.* **2024**, *34*, 462–478. [CrossRef]
18. Bell, R.; Longshore, R.; Madachy, R. *Introducing SysEngBench: A Novel Benchmark for Assessing Large Language Models in Systems Engineering*; Acquisition Research Program: Monterey, CA, USA, 2024. Available online: <https://dair.nps.edu/handle/123456789/5135> (accessed on 8 January 2026).
19. Johns, B.; Carroll, K.; Medina, C.; Lewark, R.; Walliser, J. AI Systems Modeling Enhancer (AI-SME): Initial Investigations into a ChatGPT-enabled MBSE Modeling Assistant. *INCOSE Int. Symp.* **2024**, *34*, 1149–1168. [CrossRef]
20. Chami, M.; Zoghbi, C.; Bruel, J.-M. A First Step towards AI for MBSE: Generating a Part of SysML Models from Text Using AI. In *INCOSE Artificial Intelligence: 2019 Conference Proceedings*, 1st ed.; Independently published: Chicago, IL, USA, 2019; pp. 123–136. ISBN 9781702233811. Available online: https://www.researchgate.net/publication/337338933_A_First_Step_towards_AI_for_MBSE_Generating_a_Part_of_SysML_Models_from_Text_Using_AI (accessed on 8 January 2026).
21. Poulsen, V.V.; Guertler, M.; Eisenbart, B.; Sick, N. Advancing systems engineering with artificial intelligence: A review on the future potential challenges and pathways. *Proc. Des. Soc.* **2025**, *5*, 359–368. [CrossRef]
22. Manno, N. Accelerating Semantic Digital Thread User Queries Using LLMs 2024. Available online: <https://sercuarc.org/wp-content/uploads/2025/06/Slides-Accelerating-Semantic-Digital-Thread-User-Queries-Using-LLMs.pdf> (accessed on 8 January 2026).
23. Pennock, M.; Ponnock, J.; Bannon, T.; Dahmann, J. AI-Enabled Mission Engineering. 2025. Available online: https://sercuarc.org/wp-content/uploads/2025/09/Pennock_AI_Enabled_ME.pdf (accessed on 8 January 2026).
24. Caldarini, G.; Jaf, S.; McGarry, K. A Literature Survey of Recent Advances in Chatbots. *Information* **2022**, *13*, 41. [CrossRef]
25. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*; Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M.F., Lin, H., Eds.; Curran Associates, Inc.: New York, NY, USA, 2020; pp. 1–16.
26. Sharma, C. Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers. *arXiv* **2025**, arXiv:2506.00054.
27. Edge, D.; Trinh, H.; Cheng, N.; Bradley, J.; Chao, A.; Mody, A.; Truitt, S.; Metropolitansky, D.; Ness, R.O.; Larson, J. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. 2024. Available online: <http://arxiv.org/pdf/2404.16130v2> (accessed on 8 January 2026).
28. Han, H.; Ma, L.; Shomer, H.; Wang, Y.; Lei, Y.; Guo, K.; Hua, Z.; Long, B.; Liu, H.; Aggarwal, C.C.; et al. RAG vs. GraphRAG: A Systematic Evaluation and Key Insights. *arXiv* **2025**, arXiv:2502.11371. [CrossRef]
29. Peng, B.; Zhu, Y.; Liu, Y.; Bo, X.; Shi, H.; Hong, C.; Zhang, Y.; Tang, S. Graph Retrieval-Augmented Generation: A Survey. 2024. Available online: <http://arxiv.org/pdf/2408.08921v2> (accessed on 8 January 2026).
30. Jiang, X.; Zhang, R.; Xu, Y.; Qiu, R.; Fang, Y.; Wang, Z.; Tang, J.; Ding, H.; Chu, X.; Zhao, J.; et al. HyKGE: A Hypothesis Knowledge Graph Enhanced Framework for Accurate and Reliable Medical LLMs Responses. *arXiv* **2023**, arXiv:2312.15883.
31. Li, S.; Gao, Y.; Jiang, H.; Yin, Q.; Li, Z.; Yan, X.; Zhang, C.; Yin, B. Graph Reasoning for Question Answering with Triplet Retrieval. *arXiv* **2023**, arXiv:2305.18742. [CrossRef]
32. He, X.; Tian, Y.; Sun, Y.; Chawla, N.V.; Laurent, T.; LeCun, Y.; Bresson, X.; Hooi, B. G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering. *Adv. Neural Inf. Process. Syst.* **2024**, *37*, 132876–132907.

33. Hu, Y.; Lei, Z.; Zhang, Z.; Pan, B.; Ling, C.; Zhao, L. GRAG: Graph Retrieval-Augmented Generation. *arXiv* **2024**, arXiv:2405.16506. [[CrossRef](#)]
34. Sarmah, B.; Hall, B.; Rao, R.; Patel, S.; Pasquali, S.; Mehta, D. HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction. *arXiv* **2024**, arXiv:2408.04948. [[CrossRef](#)]
35. Delile, J.; Mukherjee, S.; van Pamel, A.; Zhukov, L. Graph-Based Retriever Captures the Long Tail of Biomedical Knowledge. *arXiv* **2024**, arXiv:2402.12352. [[CrossRef](#)]
36. Jiang, J.; Zhou, K.; Dong, Z.; Ye, K.; Zhao, W.X.; Wen, J.-R. StructGPT: A General Framework for Large Language Model to Reason over Structured Data. *arXiv* **2023**, arXiv:2305.09645. [[CrossRef](#)]
37. Mavromatis, C.; Karypis, G. GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning. *arXiv* **2024**, arXiv:2405.20139. [[CrossRef](#)]
38. Gutiérrez, B.J.; Shu, Y.; Gu, Y.; Yasunaga, M.; Su, Y. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. *Adv. Neural Inf. Process. Syst.* **2024**, *37*, 59532–59569.
39. Sun, H.; Bedrax-Weiss, T.; Cohen, W.W. *PullNet: Open Domain Question Answering with Iterative Retrieval on Knowledge Bases and Text*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2019.
40. Jiang, J.; Zhou, K.; Zhao, W.X.; Song, Y.; Zhu, C.; Zhu, H.; Wen, J.-R. *KG-Agent: An Efficient Autonomous Agent Framework for Complex Reasoning over Knowledge Graph*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2024.
41. Jin, B.; Xie, C.; Zhang, J.; Roy, K.K.; Zhang, Y.; Li, Z.; Li, R.; Tang, X.; Wang, S.; Meng, Y.; et al. Graph Chain-of-Thought: Augmenting Large Language Models by Reasoning on Graphs. *arXiv* **2024**, arXiv:2404.07103. [[CrossRef](#)]
42. Sun, L.; Tao, Z.; Li, Y.; Arakawa, H. ODA: Observation-Driven Agent for integrating LLMs and Knowledge Graphs. *arXiv* **2024**, arXiv:2404.07677. [[CrossRef](#)]
43. Wang, X.; Yang, Q.; Qiu, Y.; Liang, J.; He, Q.; Gu, Z.; Xiao, Y.; Wang, W. KnowledGPT: Enhancing Large Language Models with Retrieval and Storage Access on Knowledge Bases. *arXiv* **2023**, arXiv:2308.11761. [[CrossRef](#)]
44. Munikoti, S.; Acharya, A.; Wagle, S.; Horawalavithana, S. ATLANTIC: Structure-Aware Retrieval-Augmented Language Model for Interdisciplinary Science. *arXiv* **2023**, arXiv:2311.12289.
45. Li, Z.; Guo, Q.; Shao, J.; Song, L.; Bian, J.; Zhang, J.; Wang, R. Graph Neural Network Enhanced Retrieval for Question Answering of LLMs. *arXiv* **2024**, arXiv:2406.06572.
46. Yang, R.; Liu, H.; Marrese-Taylor, E.; Zeng, Q.; Ke, Y.H.; Li, W.; Cheng, L.; Chen, Q.; Caverlee, J.; Matsuo, Y.; et al. KG-Rank: Enhancing Large Language Models for Medical QA with Knowledge Graphs and Ranking Techniques. *arXiv* **2024**, arXiv:2403.05881. [[CrossRef](#)]
47. Peng, Z.; Yang, Y. Connecting the Dots: Inferring Patent Phrase Similarity with Retrieved Phrase Graphs. *arXiv* **2024**, arXiv:2403.16265. [[CrossRef](#)]
48. Tewari, A.; Dixit, S.; Sahni, N.; Bordas, S.P. Machine learning approaches to identify and design low thermal conductivity oxides for thermoelectric applications. *Data Centric Eng.* **2020**, *1*, e8. [[CrossRef](#)]
49. Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv* **2022**, arXiv:2210.03629.
50. together.ai. Llama 3.3 70B API. Available online: <https://www.together.ai/models/llama-3-3-70b> (accessed on 17 November 2025).
51. Google. Googel AI Studio. Available online: <https://aistudio.google.com/> (accessed on 17 November 2025).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.